

Gestão de Assistências

Manual da aplicação · Docker · Cloudflare

Documentação de referência

Manual da Aplicação — Gestão de Assistências

> Documento de referência. Se um dia precisar de reinstalar, mudar de computador ou pedir ajuda a outra pessoa, está aqui tudo o que é preciso.

Índice 1. O que é e o que faz 2. Como está construída (visão técnica) 3. Estrutura de ficheiros 4. Configuração (variáveis de ambiente) 5. Pôr a correr no Docker 6. A base de dados 7. Backups 8. Mudar de máquina / migrar sem perder nada 9. Tarefas do dia a dia 10. Resolução de problemas 11. Acesso externo (Cloudflare)

---

1. O que é e o que faz

Aplicação web para gerir as assistências técnicas e os clientes de uma pequena empresa. Funciona num browser (computador ou telemóvel).

Funcionalidades: - **Clientes** — lista, pesquisa, ficha individual com histórico, criação/edição. Não deixa apagar um cliente que tenha assistências associadas (evita perder histórico). - **Assistências** — fluxo de estados: `Pendente → Em andamento → Aguarda Verificação → Concluída` (ou `Cancelada`). - **Em Aberto / Aguarda Verificação / Histórico** — cada lista mostra só as assistências no estado certo. - **Histórico** — com duas vistas: lista normal, ou **agrupado por cliente** (clica-se no cliente para abrir todo o histórico dele). - **Encomendas** — material a encomendar/receber (`Por encomendar → Já encomendado → Recebido`). - **Dashboard** — cartões de resumo, alertas (urgentes, atrasadas, por pagar, encomendas) e 6 gráficos (estados, evolução mensal, tipos, idade das abertas, pagas vs por pagar, top clientes). - **Pesquisa** em todas as listas; **paginação** com 10/20/50 por página. - **Login** com utilizador e palavra-passe.

---

2. Como está construída (visão técnica)

Camada	Tecnologia
Linguagem	Python 3.12
Framework web	Flask
Servidor de produção	gunicorn
Base de dados	SQLite (ficheiro único)
Interface	HTML + Bootstrap 5 + Chart.js (servido localmente)
Empacotamento	Docker / Docker Compose
Acesso externo	Cloudflare Tunnel (opcional)

Toda a lógica está num único ficheiro, `app.py` : rotas, acesso à base de dados, validações e segurança (login, CSRF, cabeçalhos).

---

3. Estrutura de ficheiros

```
ProjetoWeb/
├── app.py                      ← toda a lógica da aplicação
├── requirements.txt            ← dependências Python
├── Dockerfile                  ← como se constrói a imagem
├── docker-compose.yml          ← arranque LOCAL (sem túnel)
├── docker-compose.prod.yml     ← arranque PRODUÇÃO/EXTERNO (com túnel)
├── Dockerfile.backup           ← contentor de backup automático
├── backup.sh                   ← script de backup diário da BD
├── .env                        ← SEGREDOS (não partilhar!) — criado por si
├── .env.example                ← modelo do .env (sem segredos)
├── .dockerignore / .gitignore
├── templates/                  ← páginas HTML
├── static/                     ← css, javascript, Chart.js
├── database/
│   ├── assistencias.db         ← ★ A BASE DE DADOS (os seus dados)
│   └── backups/                ← cópias de segurança automáticas
├── instance/
│   └── secret_key              ← chave gerada (se não usar SECRET_KEY no .env)
└── docs/                       ← esta documentação
```

O que **realmente** importa preservar: a pasta `database/` e o ficheiro `.env`. O resto é código, reconstrói-se.

---

4. Configuração (variáveis de ambiente)

Definem-se no `.env` (segredos) e no ficheiro compose (as restantes).

Variável	Para que serve	Local	Externo (Cloudflare)
`SECRET_KEY`	Protege as sessões. Deve ser fixa e secreta.	opcional[1]	**obrigatória**
`SESSION_COOKIE_SECURE`	Cookies só por HTTPS.	`0`	`1`
`BEHIND_PROXY`	App confia no túnel/proxy à frente.	`0`	`1`
`FLASK_DEBUG`	Modo de depuração. **Nunca `1` em produção.**	`0`	`0`
`DATABASE_PATH`	Caminho alternativo da BD (raramente usado).	—	—
`TUNNEL_TOKEN`	Token do Cloudflare Tunnel.	—	**obrigatória**

[1] Se não definir `SECRET_KEY`, a app gera uma e guarda-a em `instance/secret_key`. Como a pasta `instance/` é um volume, a chave mantém-se entre reinícios. Mesmo assim, em acesso externo **defina** uma `SECRET_KEY` própria no `.env`.

---

5. Pôr a correr no Docker

Pré-requisito: **Docker Desktop** instalado e a correr.

5.1 Modo LOCAL (só nesta máquina)

```
cd C:\Caminho\Para\ProjetoWeb
docker compose up -d --build
```

Aceder em **http://localhost:5000**.

5.2 Modo EXTERNO (com Cloudflare)

1. Crie o ficheiro `.env` (copie de `.env.example`) com `SECRET_KEY` e `TUNNEL_TOKEN`.
2. Arranque com o compose de produção:

```
docker compose -f docker-compose.prod.yml up -d --build
```

Detalhes completos do Cloudflare em CLOUDFLARE.md.

5.3 Comandos úteis

```
docker compose ps                # ver estado dos contentores
docker compose logs -f web       # ver registos da app (Ctrl+C para sair)
docker compose logs -f tunnel    # ver registos do túnel
docker compose down              # parar (NÃO apaga a base de dados)
docker compose up -d --build     # reconstruir e arrancar
docker compose restart web       # reiniciar só a app
```

> ⚠️ **Nunca** use `docker compose down -v`. O `-v` apaga volumes. (Mesmo assim, como a BD está numa pasta do disco e não num volume anónimo, estaria protegida — mas é boa prática evitar o `-v`.)

5.4 Primeiro acesso

- Utilizador: `admin` · Palavra-passe inicial: `admin123`
- A app avisa no login enquanto esta palavra-passe não for mudada.
- **Mude-a já** no ícone ⚙ (Configuração).

---

6. A base de dados

- É **um único ficheiro**: `database/assistencias.db` (formato SQLite).
- A pasta `database/` está montada como **volume** no Docker — ou seja, os dados vivem no disco do Windows, **fora** da imagem. Reconstruir a imagem (`--build`) **não** toca nos dados.
- Ao arrancar, a app cria tabelas em falta e aplica **migrações aditivas** (só acrescenta colunas/normaliza valores, nunca apaga). Antes de qualquer alteração de estrutura, **cria automaticamente um backup** em `database/backups/`.

Conteúdo (à data desta documentação)

Tabelas: `clientes`, `assistencias`, `encomendas`, `usuarios`.

---

7. Backups

A regra de ouro: **fazer cópia do ficheiro** `database/assistencias.db`.

Backup manual (rápido)

```
Copy-Item C:\Caminho\Para\ProjetoWeb\database\assistencias.db C:\backup-banco\
```

Backup com data no nome

```
Copy-Item database\assistencias.db "C:\backup-banco\assistencias-$(Get-Date -Format 'yyyyMMdd').db"
```

Backup AUTOMÁTICO (contentor dedicado)

O projeto inclui um contentor `backup` que faz uma cópia da base de dados **todos os dias às 03:00** (e também uma assim que arranca). É definido em `Dockerfile.backup` + `backup.sh` e já está incluído nos dois ficheiros compose.

- **Onde ficam as cópias:** `database/backups/auto/` (no disco do Windows), com nome `assistencias-AAAAMMDD-HHMMSS.db.gz` (comprimidas).
- **Cópia segura:** usa o `.backup` do SQLite, que produz uma cópia íntegra mesmo com a aplicação a escrever ao mesmo tempo.
- **Rotação:** apaga sozinho as cópias com mais de **30 dias** (configurável na variável `RETENCAO_DIAS` no compose).

Para o ativar, basta (re)construir os contentores:

```
docker compose -f docker-compose.prod.yml up -d --build   # produção
# ou, em local:  docker compose up -d --build
```

Confirmar que está a correr e ver o registo:

```
docker compose -f docker-compose.prod.yml logs backup --tail 20
```

Mudar a frequência (ex.: semanal, domingos às 3h): edite a linha do cron em `Dockerfile.backup` — `0 3 * * *` (diário) para `0 3 * * 0` (domingos) — e reconstrua. **Mudar para quantos dias guardar:** altere `RETENCAO_DIAS` no compose.

Restaurar a partir de um backup

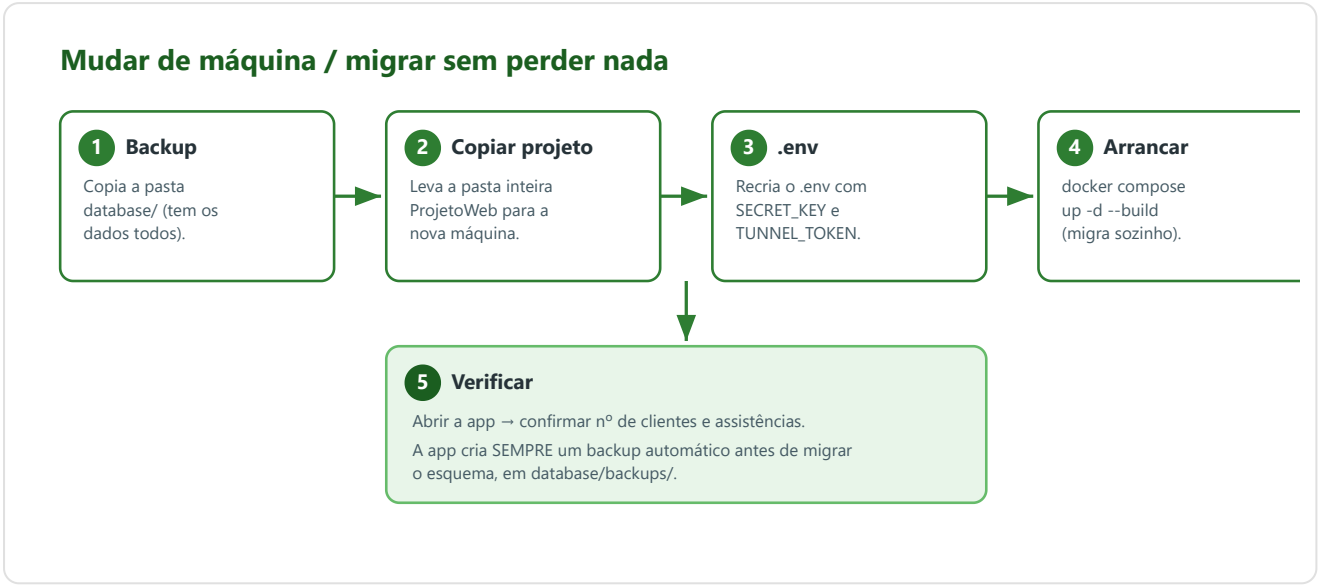
```
# 1. parar a app
docker compose -f docker-compose.prod.yml down
# 2. descomprimir a cópia escolhida para o lugar da BD ativa
cd C:\Caminho\Para\ProjetoWeb\database
# (no Windows, clique direito no .gz e extraia, ou use o 7-Zip)
#   renomeie o ficheiro extraído para "assistencias.db"
# 3. arrancar de novo
cd ..
docker compose -f docker-compose.prod.yml up -d --build
```

Recomendações

- O backup automático fica **na mesma máquina**. Para proteção real contra avaria do disco/roubo, copie **periodicamente** a pasta `database/backups/auto/` (ou o `assistencias.db`) para **outro local** (disco externo, NAS, cloud).
- Guarde também uma cópia do ficheiro `.env` num sítio seguro (gestor de palavras-passe). Sem ele perde a `SECRET_KEY` e o `TUNNEL_TOKEN`.
- A pasta `database/backups/` também guarda os backups das **migrações** (feitos pela app antes de alterar o esquema).

---

8. Mudar de máquina / migrar sem perder nada



1. **Backup**: copie a pasta `database/` (e o `.env`) para um sítio seguro.
2. **Copiar o projeto**: leve a pasta inteira `ProjetoWeb` para a nova máquina (com a pasta `database/` lá dentro).
3. `.env`: recrie o `.env` com a `SECRET_KEY` e o `TUNNEL_TOKEN` (ou copie o original).
4. **Arrancar**: `docker compose up -d --build` (local) ou `docker compose -f docker-compose.prod.yml up -d --build` (externo). A app adapta o esquema sozinha e cria backup antes.
5. **Verificar**: abra a app e confirme o número de clientes e assistências no Dashboard.

> Se tiver duas bases de dados e não souber qual é a mais recente, **não apague nenhuma**. Abra cada uma e compare as datas das últimas assistências — ou peça ajuda antes de substituir.

---

9. Tarefas do dia a dia

Quero…	Onde
Mudar a palavra-passe	⚙ Configuração
Criar cliente	Clientes → «Novo Cliente»
Criar assistência	Em Aberto → «Nova Assistência» (ou a partir da ficha do cliente)
Marcar trabalho feito	botão ✓ na lista «Em Aberto»
Aprovar / marcar paga	página «Aguarda Verificação»
Ver histórico de um cliente	Histórico → «Por cliente», ou ficha do cliente
Gerir material	«Encomendas»

---

10. Resolução de problemas

Sintoma	Causa provável	Solução
`invalid interpolation format … ${...}`	Pôs a chave diretamente no compose em vez do nome	No compose fica `${SECRET_KEY}`; o valor vai no `.env`.
Não consigo fazer login externamente	`SESSION_COOKIE_SECURE=1` sem HTTPS	Em acesso externo tem de haver HTTPS (Cloudflare). Localmente use `0`.
Porta 5000 ocupada	Outro processo/contentor a usar a porta	`docker compose down` no projeto antigo; ou ver quem a usa.
App «desatualizada» no browser	Cache	Recarregar com Ctrl+F5.
O túnel não liga	Token errado/expirado	Ver `docker compose logs tunnel`; renovar token no Cloudflare.
Quero ver o que se passa	—	`docker compose logs -f web`

Rollback (voltar a só-local)

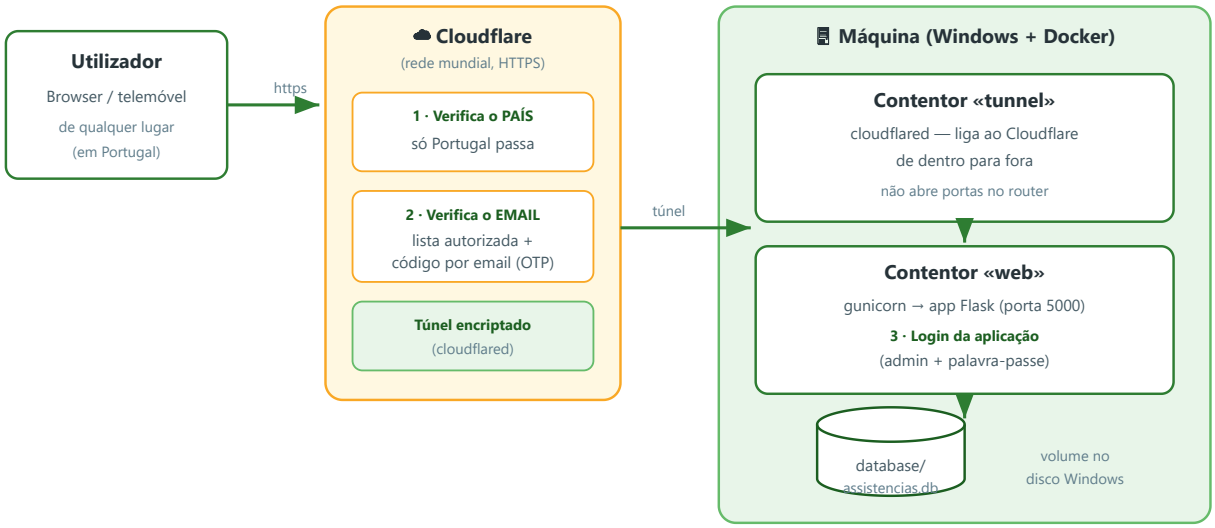
Se o acesso externo causar problemas, volte ao modo local sem perder dados:

```
docker compose -f docker-compose.prod.yml down
docker compose up -d --build
```

11. Acesso externo (Cloudflare)

Toda a configuração de domínio, túnel, autorização por email e bloqueio por país está no guia dedicado: **CLOUDFLARE.md**.

Arquitetura — acesso externo seguro



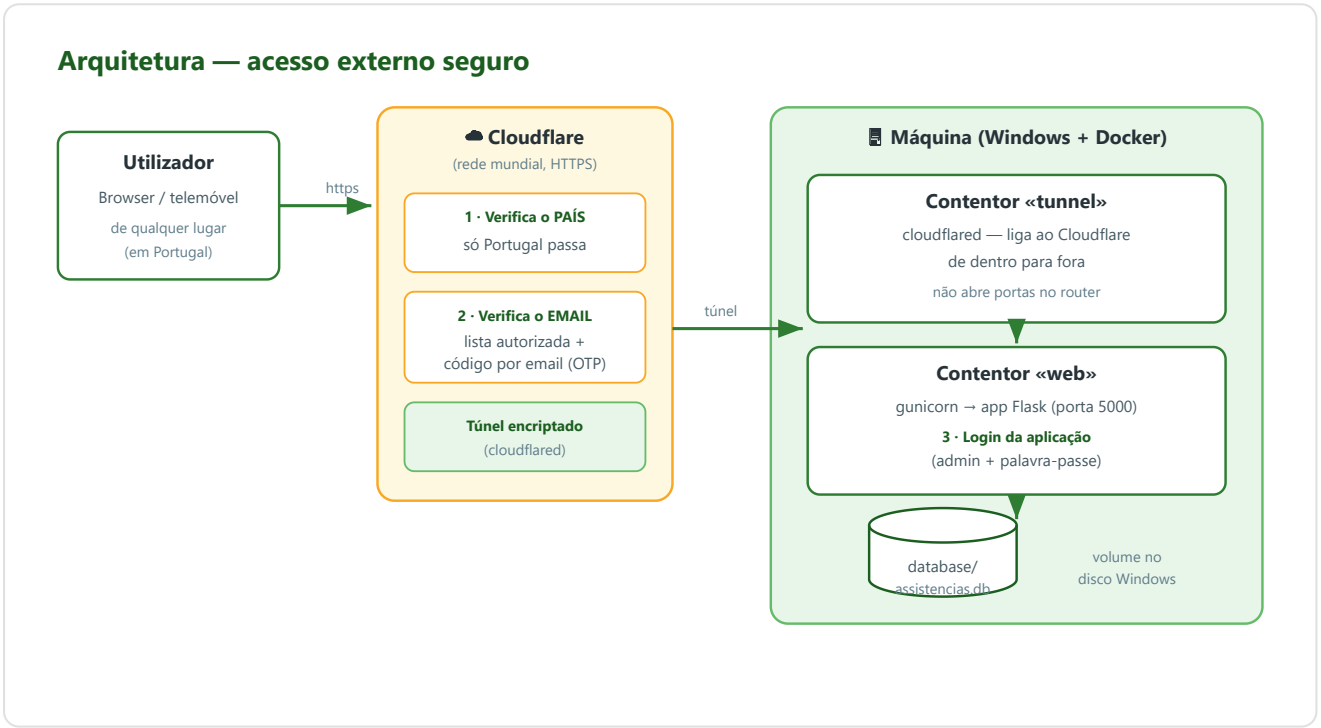
Guia Cloudflare — Acesso externo seguro

> Como tornar a aplicação acessível de fora (Internet) com **HTTPS**, sem abrir portas no router, e protegida por **três camadas**: país, email e login da aplicação.

Índice 1. Como funciona (resumo) 2. O domínio no Cloudflare 3. Criar o Túnel 4. Ligar a aplicação ao túnel (Docker) 5. Mapear o endereço público 6. Proteção por email + país (Cloudflare Access) 7. Gestão do dia a dia 8. Segurança máxima — checklist 9. Resolução de problemas

---

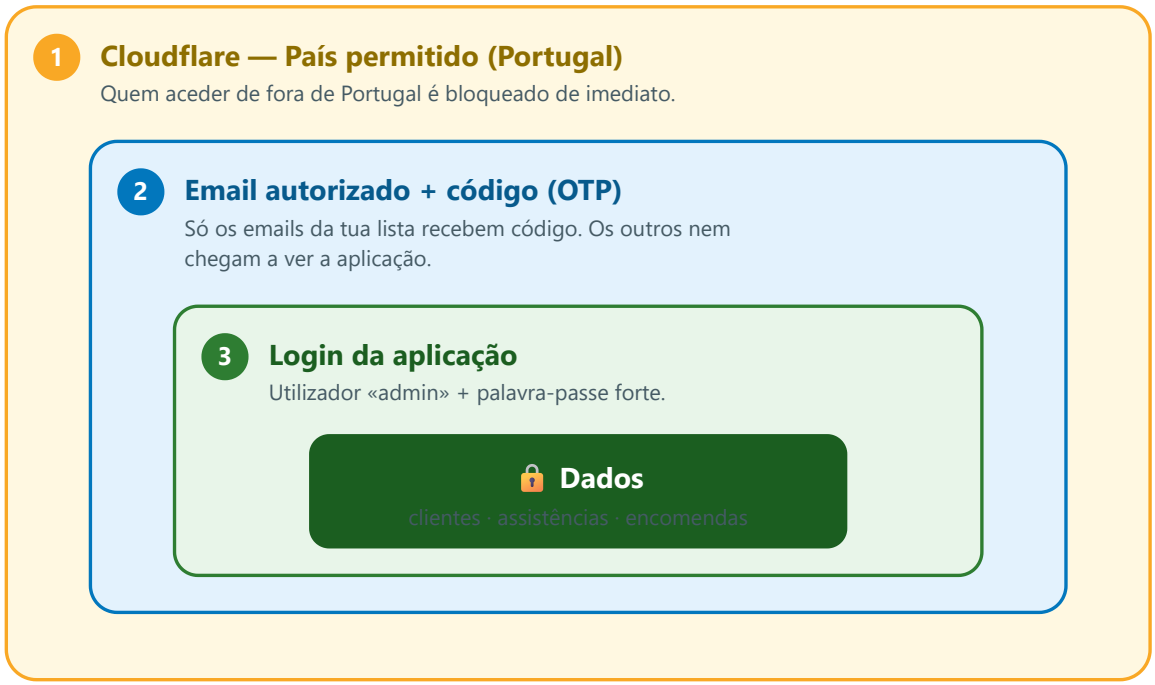
1. Como funciona (resumo)



Quando alguém abre `https://assistencias.oseudominio.pt`:

1. O pedido chega ao **Cloudflare** (não diretamente à sua máquina).
2. O Cloudflare verifica o **país** → só Portugal passa.
3. Verifica o **email** → só os da lista; envia um **código** para o email.
4. Só então o pedido entra pelo **túnel encriptado** até ao contentor Docker.
5. Por fim, a pessoa ainda tem de fazer **login na aplicação**.

As camadas de segurança (do exterior para o núcleo)



Para um atacante chegar aos dados teria de furar as três camadas. Cada uma é independente das outras.

Vantagens: HTTPS automático, sem abrir portas no router, e a máquina nunca fica diretamente exposta à Internet.

---

2. O domínio no Cloudflare

Precisa de um domínio gerido pelo Cloudflare (plano **Free** chega).

- **Não tem domínio?** Em https://dash.cloudflare.com → secção **Registrar** → compre um (~10 €/ano). Fica logo configurado no Cloudflare.
- **Já tem domínio noutro registador?** No Cloudflare: **Add a site** → escreva o domínio → escolha o plano **Free** → o Cloudflare dá-lhe **2 nameservers**. Vá ao painel do registador onde comprou o domínio e substitua os nameservers pelos do Cloudflare. (Pode demorar de minutos a algumas horas a ficar ativo.)

Quando o domínio aparecer como **Active** no Cloudflare, pode continuar.

---

3. Criar o Túnel

O túnel é a "ponte" segura entre o Cloudflare e a sua máquina.

1. Em https://one.dash.cloudflare.com (painel **Zero Trust**).
2. Na primeira utilização pode pedir para escolher um plano — escolha o **Free** (pode pedir um cartão para ativar a conta, mas não é cobrado).
3. Menu lateral: **Networks** → **Tunnels** → **Create a tunnel**.
4. Tipo de conector: **Cloudflared** → **Next**.
5. Dê um nome ao túnel, por exemplo `assistencias` → **Save tunnel**.

6. No ecrã "Install and run a connector", escolha o separador **Docker**. **Copie apenas o token** — a longa sequência de caracteres a seguir a `--token`. **Não** corra o comando que eles mostram (vamos usar o nosso compose). Guarde o token num sítio seguro.

> O token é um segredo. É o que autoriza a sua máquina a ligar-se ao túnel.

---

4. Ligar a aplicação ao túnel (Docker)

Na pasta do projeto:

4.1 Gerar a SECRET_KEY (uma vez)

```
docker run --rm python:3.12-slim python -c "import secrets; print(secrets.token_hex(32))"
```

4.2 Criar o ficheiro `.env`

Crie um ficheiro chamado `.env` (a partir do `.env.example`) com:

```
SECRET_KEY=cole-aqui-a-chave-gerada-no-passo-4.1
TUNNEL_TOKEN=cole-aqui-o-token-copiado-do-cloudflare
```

> ⚠️ **Importante:** no ficheiro **compose** ficam os *nomes* das variáveis (`${SECRET_KEY}` , `${TUNNEL_TOKEN}`). Os *valores* ficam **só** no `.env` . Foi exatamente este o erro do "invalid interpolation": pôr o valor onde devia estar o nome.

4.3 Arrancar com o compose de produção

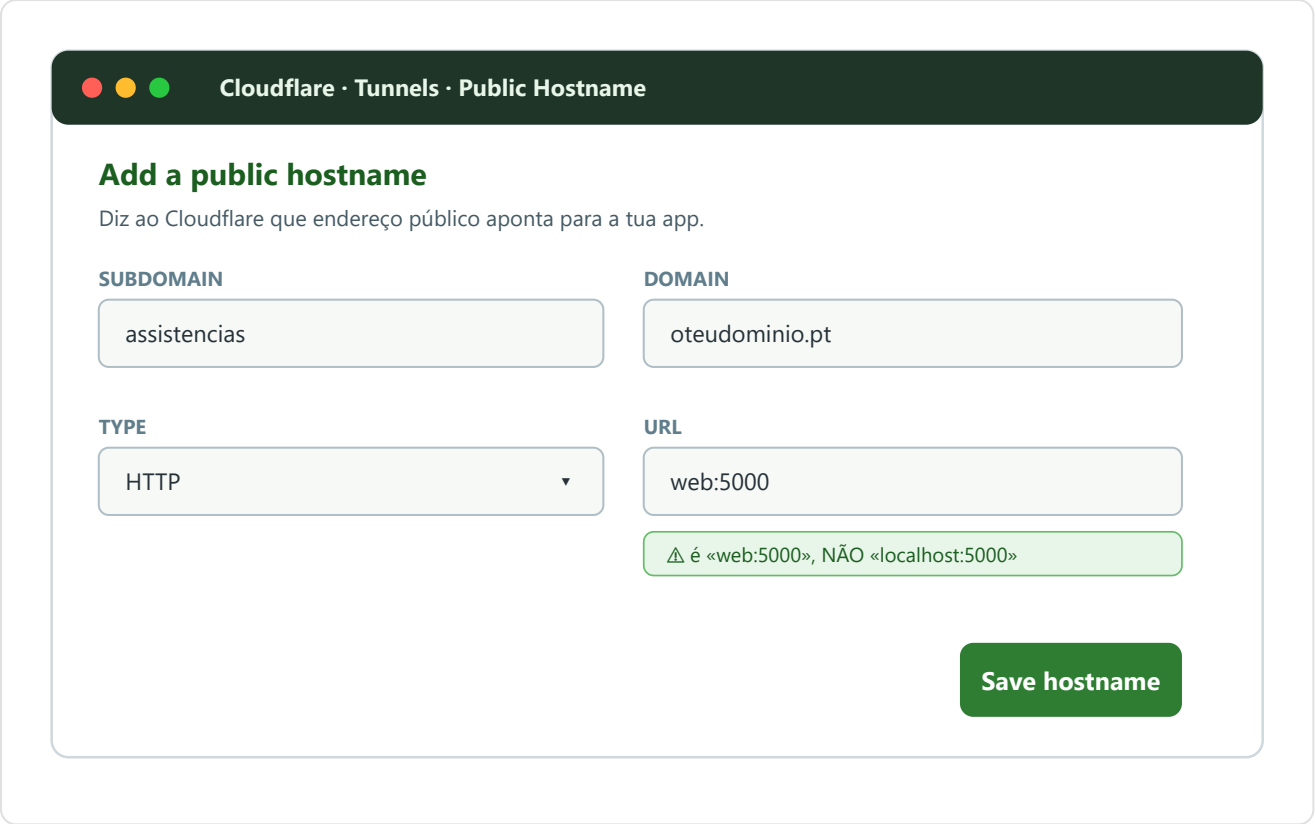
```
cd C:\Caminho\Para\ProjetoWeb
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml logs tunnel --tail 20
```

Nos registos do `tunnel` deve aparecer `Registered tunnel connection`. No painel do Cloudflare, o túnel passa a **HEALTHY**.

---

5. Mapear o endereço público

Diz ao Cloudflare que endereço da Internet aponta para a aplicação.



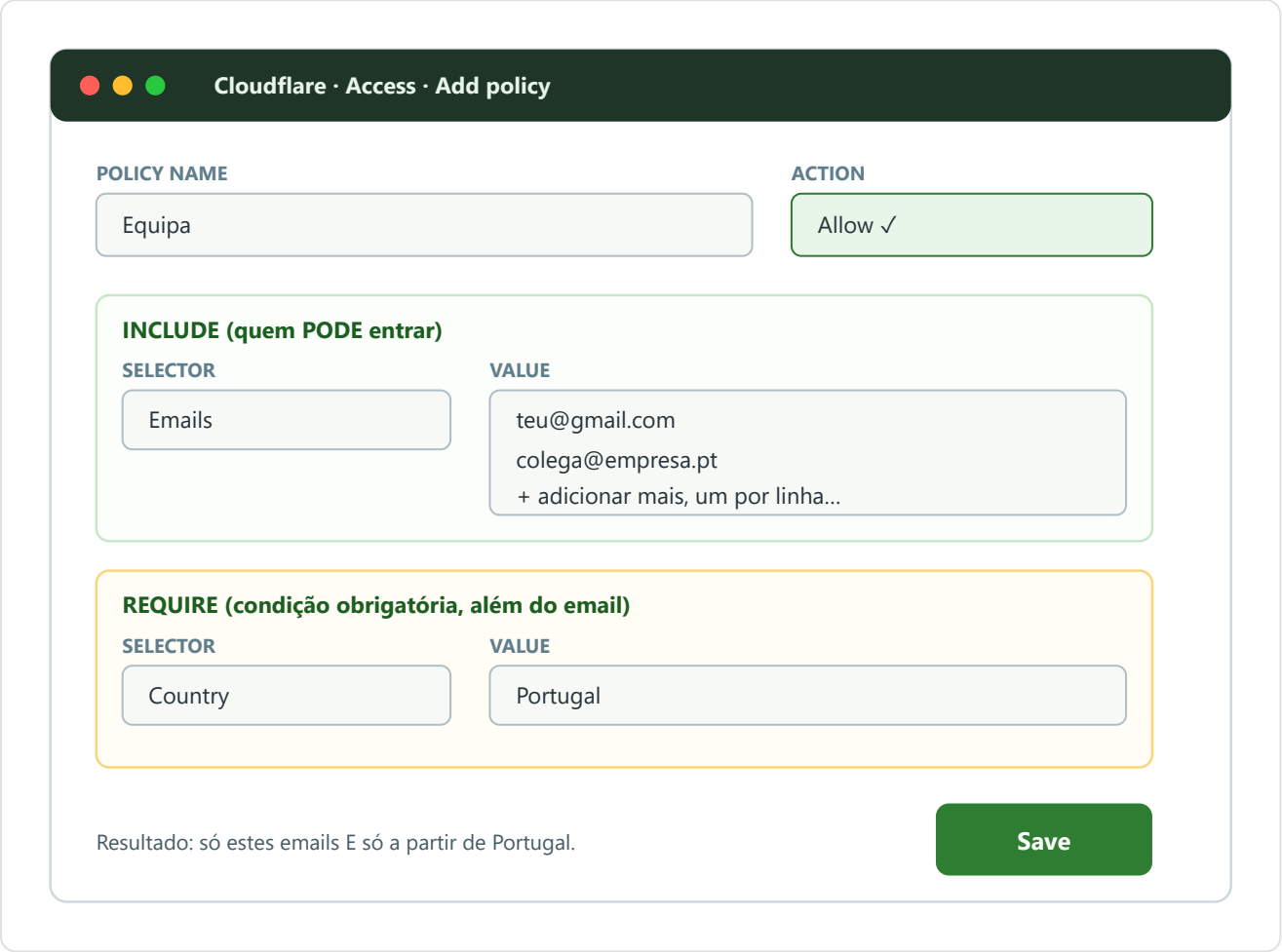
1. **Networks** → **Tunnels** → abra o seu túnel → separador **Public Hostname** → **Add a public hostname**.
2. Preencha:
3. **Subdomain**: `assistencias`
4. **Domain**: `oseudominio.pt`
5. **Type**: `HTTP`
6. **URL**: `web:5000` ← é o nome do serviço no Docker, **não** `localhost`
7. **Save hostname**. O registo de DNS é criado automaticamente.

A partir daqui, `https://assistencias.oseudominio.pt` já chega à aplicação (com HTTPS). Falta só proteger quem pode entrar.

---

6. Proteção por email + país (Cloudflare Access)

Esta é a parte que controla **quem** pode aceder. **Só o administrador da conta Cloudflare** (você) gere esta lista — ninguém entra só por escrever um email; tem de estar na lista que você define.



6.1 Criar a aplicação protegida

1. Painel **Zero Trust** → **Access** → **Applications** → **Add an application**.
2. Tipo: **Self-hosted**.
3. Preencha:
4. **Application name**: `Gestao de Assistencias`
5. **Session Duration**: `24 hours` (quanto tempo fica autenticado sem repetir o código)
6. **Application domain / Public hostname**: `assistencias` + `oseudominio.pt` — **tem de ser o mesmo endereço** do passo 5.
7. **Next**.

6.2 Criar a política (emails autorizados + país)

No passo das **Policies** (dentro do assistente; em alguns painéis é uma secção à parte chamada **Access** → **Policies** — os campos são os mesmos):

1. **Policy name**: `Equipa`
2. **Action**: `Allow`
3. **Include** (quem **pode** entrar):
4. **Selector**: `Emails`
5. **Value**: escreva cada email autorizado, ex.: `voce@gmail.com` , `colega@empresa.pt`
6. Esta lista é a chave: **só** estes emails recebem código.
7. **Require** (condição **obrigatória**, além do email) — o bloqueio por país:
8. clique em **Add require**

9. **Selector**: `Country`
10. **Value**: `Portugal`
11. Assim, mesmo um email da lista só entra **a partir de Portugal**.
12. Guardar a política → **Next** → **Add application**.

> **Include** vs **Require**: - **Include** = "basta cumprir uma destas" (a lista de emails). - **Require** = "tem de cumprir SEMPRE esta" (estar em Portugal). Resultado combinado: **email na lista E em Portugal**.

6.3 Método de login por email

O envio do código por email (**One-time PIN**) vem **ativo por defeito** — não precisa de configurar nada. (Fica em **Settings** → **Authentication**, caso queira confirmar; o nome do menu varia entre versões do painel.)

6.4 Testar

1. Em **janela anónima** ou no **telemóvel com dados móveis**, abra `https://assistencias.oseudominio.pt`.
2. Aparece o ecrã do Cloudflare a pedir o email.
3. **Teste positivo:** email da lista → recebe código no email → introduz → vê o login da aplicação. ✅
4. **Teste negativo:** email **fora** da lista → acesso negado, sem código. ❌

---

7. Gestão do dia a dia

Onde: Zero Trust → **Access** → **Applications** → **Gestao de Assistencias** → **Policies** → **Equipa**.

Quero...	Como
Autorizar uma pessoa	Edite a política, acrescente o email em **Include** → **Emails**, **Save**. Efeito imediato.
Remover uma pessoa	Tire o email da lista e **Save**.
Cortar acesso já (mesmo com sessão aberta)	Remova o email **e** em **Applications** → **(a app)** → ⋯ → **Revoke existing tokens**.
Ver quem entrou / tentativas recusadas	Zero Trust → **Logs** → **Access**.
Mudar o tempo de sessão	Na aplicação → **Session Duration**.

---

8. Segurança máxima — checklist

Por ordem de impacto:

- [ ] **2FA na conta Cloudflare** — é a conta que controla tudo. Painel normal do Cloudflare → **My Profile** → **Authentication** → **Two-Factor**.
- [ ] **Palavra-passe forte no login da app** (mudar a `admin123` em ⚙).
- [ ] **Require Country = Portugal** na política (passo 6.2). ✅ já ativado.
- [ ] **Lista de emails individuais e nominais** (evitar regras tipo "qualquer @gmail.com"); remover quem sair da equipa.
- [ ] **Renovar o token do túnel** se alguma vez foi exposto: Networks → Tunnels → o túnel → ⋯ → **Refresh token** → atualizar o `.env` → `docker compose -f docker-compose.prod.yml up -d`.
- [ ] `SESSION_COOKIE_SECURE=1`, `BEHIND_PROXY=1`, `SECRET_KEY` **própria** no modo externo (já vêm no `docker-compose.prod.yml`).

- [ ] **Porta em** `127.0.0.1` no compose de produção (já configurado) — a app não fica exposta diretamente, só pelo túnel.
- [ ] **Backups regulares** de `database/assistencias.db` e do `.env`.

Com país + email + login, são **3 barreiras independentes** entre a Internet e os seus dados.

---

9. Resolução de problemas

Sintoma	Causa	Solução
Túnel não fica **HEALTHY**	Token errado/expirado	`docker compose -f docker-compose.prod.yml logs tunnel`; renovar token (passo 8) e atualizar `.env`.
Site dá **502 Bad Gateway**	URL do hostname errado	No passo 5, **URL** tem de ser `web:5000` (não `localhost`).
Site **não abre** / DNS	Hostname/domínio não propagado	Confirme que o domínio está **Active** no Cloudflare e o hostname criado.
Recebo código mas depois **não entra na app**	Cookies inseguros	Confirme `SESSION_COOKIE_SECURE=1` e `BEHIND_PROXY=1` no compose de produção.
Email **da lista** é recusado	Está fora de Portugal, ou a política tem o email mal escrito	Verifique o **Require Country** e a lista de emails. Veja **Logs** → **Access**.
Quero **desligar** o acesso externo	—	`docker compose -f docker-compose.prod.yml down` e volte ao `docker compose up -d` local. A app continua a funcionar em `http://localhost:5000`.

Lembrete final

O ficheiro `.env` (com `SECRET_KEY` e `TUNNEL_TOKEN`) e a conta **Cloudflare** são as "chaves do reino". Guarde-os bem e nunca os partilhe. Para revogar todo o acesso externo de uma vez, basta **apagar o túnel** no painel do Cloudflare — a aplicação continua a funcionar localmente.